

|                                                                                                                                                                                                                          |                                                                                                                                              |                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b><br>Marwadi Chandarana Group  | <b>Marwadi University</b><br><b>Faculty of Engineering &amp; Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Digital Signal and Image Processing (01CT1513)</b>                                                                                                                                                           | <b>Aim:</b> Simulate discrete time sequences.                                                                                                |                                   |
| <b>Experiment No: 1</b>                                                                                                                                                                                                  | <b>Date:</b>                                                                                                                                 | <b>Enrollment No: 92510133028</b> |

**Aim:** Simulate discrete time sequences.

```
import numpy as np
import matplotlib.pyplot as plt
def unit_impulse(length, position):
    signal = np.zeros(length)
    signal[position] = 1
    return signal
# Parameters
start = -10 # Start value of the x-axis range
stop = 10 # Stop value of the x-axis range
step = 1 # Step size
# Generate x-axis values
x = np.arange(start, stop+step, step)
# Generate unit impulse signal
impulse_signal = unit_impulse(len(x), abs(start)//step)
# Plot the signal
plt.stem(x, impulse_signal)
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.title('Unit Impulse Signal')
plt.grid(True)
plt.show()
```

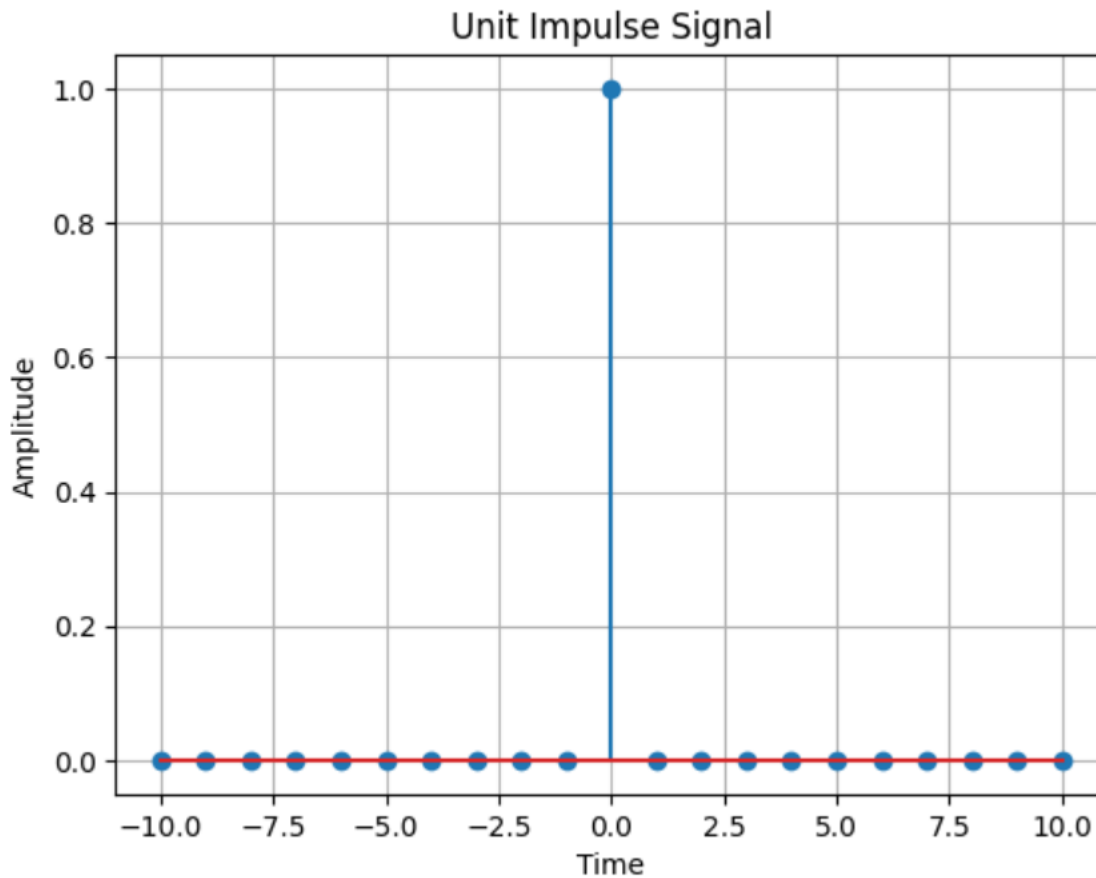
**Subject: Digital Signal and  
Image Processing (01CT1513)**

**Aim:** Simulate discrete time sequences.

**Experiment No: 1**

**Date:**

**Enrollment No: 92510133028**



```
import numpy as np
import matplotlib.pyplot as plt

def simulate_impulse_train(signal_length, period):
    impulse_train = np.zeros(signal_length)
    for n in range(signal_length):
        if n % period == 0:
            impulse_train[n] = 1
    return impulse_train

# Define the parameters for the impulse train
signal_length = 100 # Length of the impulse train
```

**Subject: Digital Signal and Image Processing (01CT1513)**

**Aim:** Simulate discrete time sequences.

**Experiment No: 1**

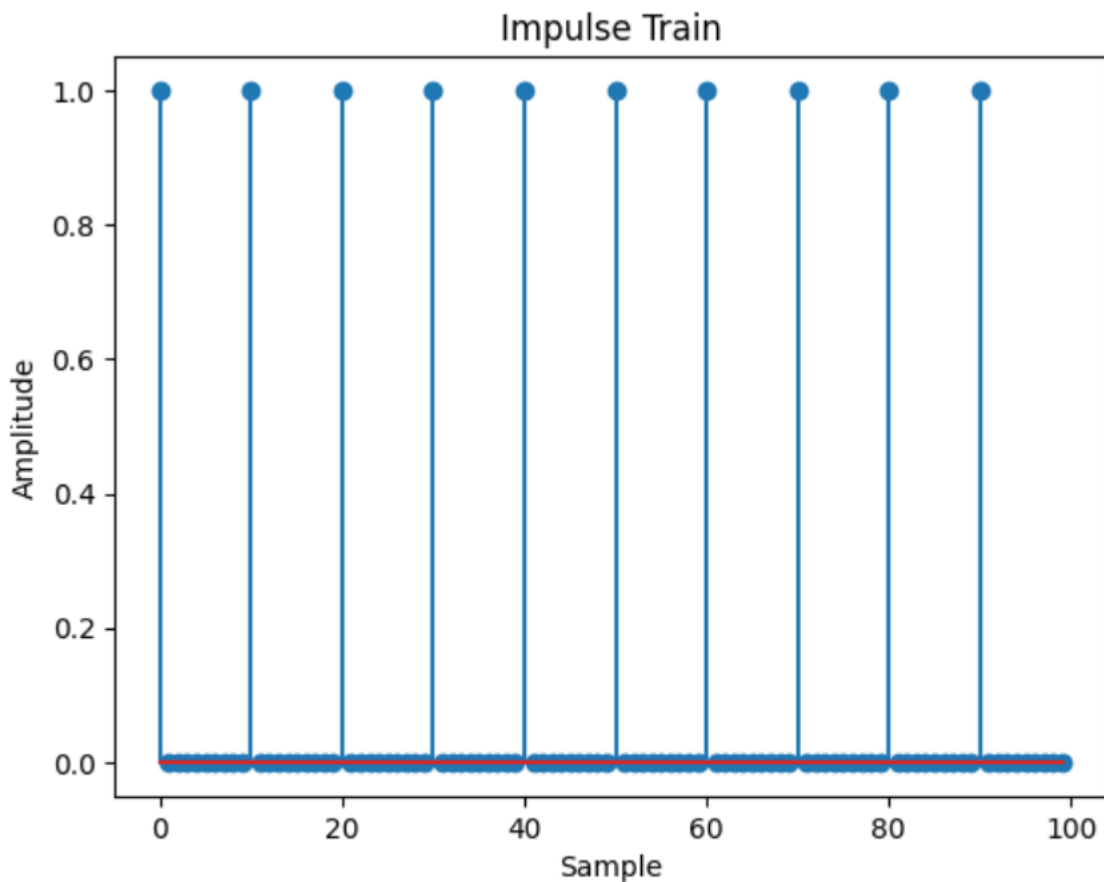
**Date:**

**Enrollment No: 92510133028**

```
period = 10 # Period of the impulse train

# Simulate the impulse train
impulse_train = simulate_impulse_train(signal_length, period)

# Plot and display the impulse train
plt.stem(impulse_train)
plt.title('Impulse Train')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.show()
```



```
# Plot and display the continuous and discrete unit step signals
```

**Subject: Digital Signal and Image Processing (01CT1513)**

**Aim:** Simulate discrete time sequences.

**Experiment No: 1**

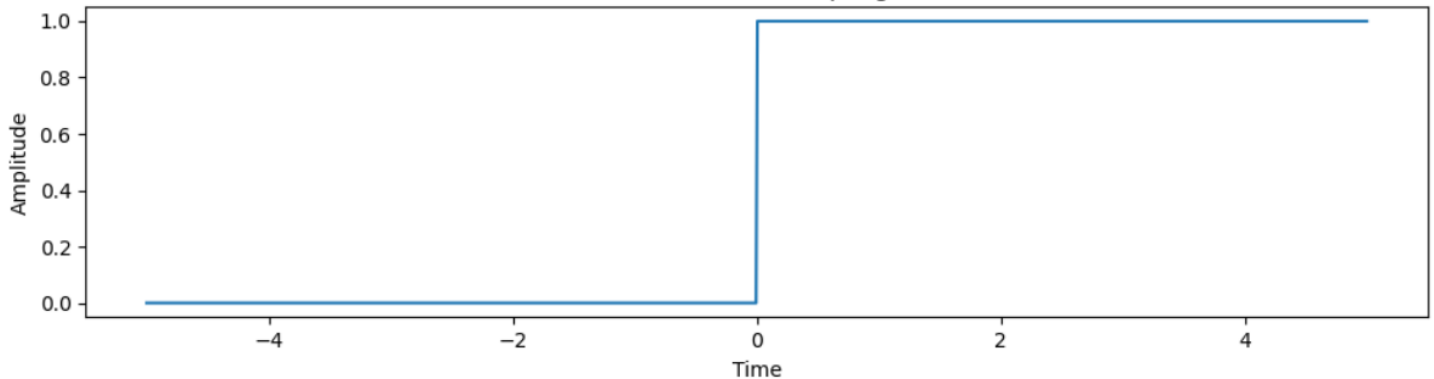
**Date:**

**Enrollment No: 92510133028**

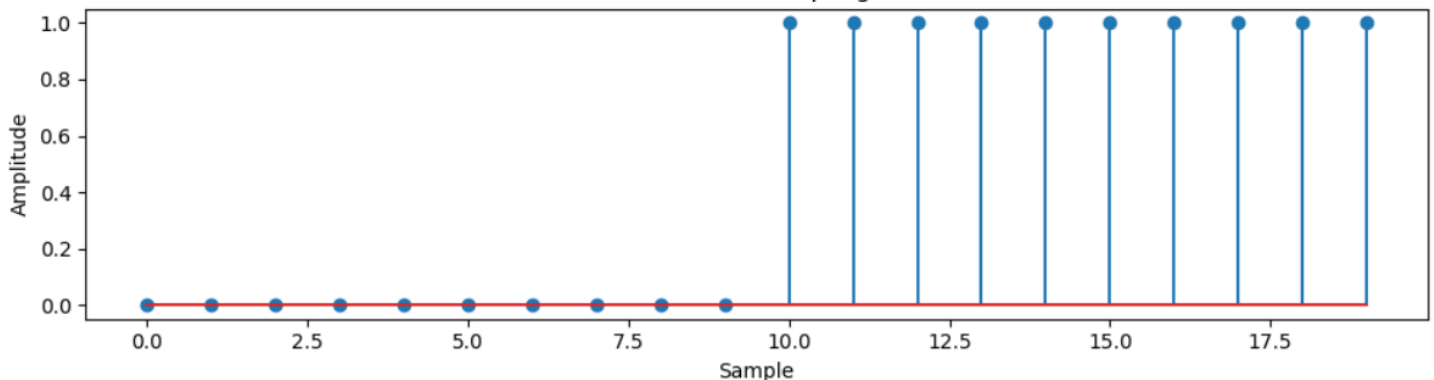
```
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_unit_step)
plt.title('Continuous Unit Step Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_unit_step)
plt.title('Discrete Unit Step Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

# Save the unit step signal arrays (optional)
# np.savetxt('continuous_unit_step.txt', continuous_unit_step, delimiter=',') #
np.savetxt('discrete_unit_step.txt', discrete_unit_step, delimiter=',')
```

Continuous Unit Step Signal



Discrete Unit Step Signal



|                                                                                                                                                                                                                          |                                                                                                                                              |                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b><br>Marwadi Chandarana Group  | <b>Marwadi University</b><br><b>Faculty of Engineering &amp; Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Digital Signal and Image Processing (01CT1513)</b>                                                                                                                                                           | <b>Aim:</b> Simulate discrete time sequences.                                                                                                |                                   |
| <b>Experiment No: 1</b>                                                                                                                                                                                                  | <b>Date:</b>                                                                                                                                 | <b>Enrollment No: 92510133028</b> |

```

import matplotlib.pyplot as plt
import numpy as np

def simulate_continuous_ramp(time, slope):
    ramp = np.zeros_like(time)
    ramp[time >= 0] = slope * time[time >= 0]
    return ramp

def simulate_discrete_ramp(num_samples, slope):
    ramp = np.zeros(num_samples)
    ramp[num_samples // 2:] = slope * np.arange(num_samples // 2, num_samples)
    return ramp

# Define the time range for the continuous ramp signal
time = np.linspace(-5, 5, 1000) # Time range from -5 to 5

# Define the number of samples and slope for the discrete ramp signal
num_samples = 20 # Number of samples
slope = 2 # Slope of the ramp

# Simulate the continuous ramp signal
continuous_ramp = simulate_continuous_ramp(time, slope)

# Simulate the discrete ramp signal
discrete_ramp = simulate_discrete_ramp(num_samples, slope)

# Plot and display the continuous and discrete ramp signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_ramp)
plt.title('Continuous Ramp Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
plt.stem(discrete_ramp)
plt.title('Discrete Ramp Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

**Subject: Digital Signal and Image Processing (01CT1513)**

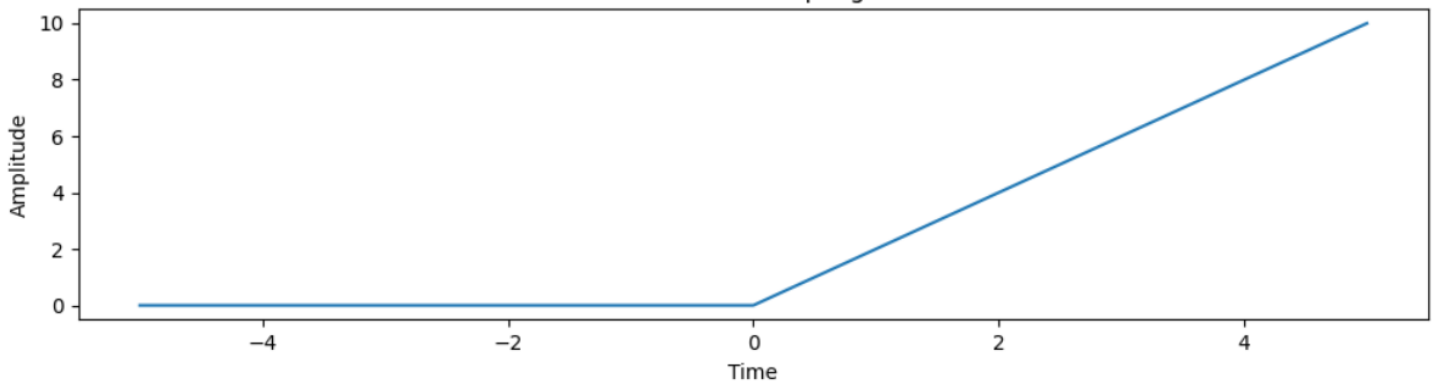
**Aim:** Simulate discrete time sequences.

**Experiment No: 1**

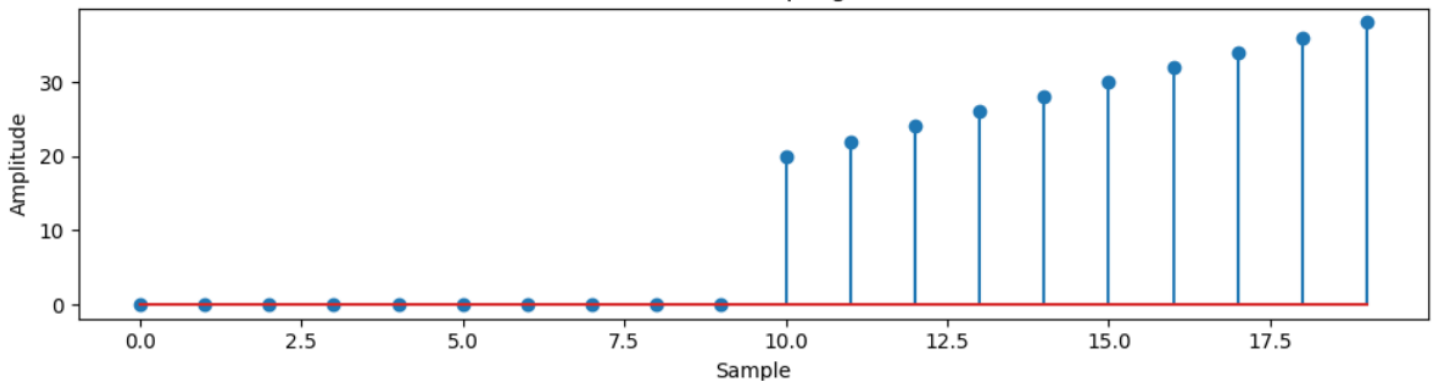
**Date:**

**Enrollment No: 92510133028**

Continuous Ramp Signal



Discrete Ramp Signal



```
import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_exponential(time, amplitude, coefficient):
    exponential_signal = amplitude * np.exp(coefficient * time)
    return exponential_signal

def simulate_discrete_exponential(num_samples, amplitude, coefficient):
    exponential_signal = amplitude * np.exp(coefficient * np.arange(num_samples))
    return exponential_signal

# Define the time range for the continuous exponential signal
time = np.linspace(0, 5, 1000) # Time range from 0 to 5

# Define the number of samples, initial amplitude, and coefficient for the discrete
exponential signal
```

|                                                                                                                                                                                                                          |                                                                                                                                              |                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b><br>Marwadi Chandarana Group  | <b>Marwadi University</b><br><b>Faculty of Engineering &amp; Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Digital Signal and Image Processing (01CT1513)</b>                                                                                                                                                           | <b>Aim:</b> Simulate discrete time sequences.                                                                                                |                                   |
| <b>Experiment No: 1</b>                                                                                                                                                                                                  | <b>Date:</b>                                                                                                                                 | <b>Enrollment No: 92510133028</b> |

```

num_samples = 20 # Number of samples
amplitude = 2 # Initial amplitude
coefficient = -0.5 # Exponential coefficient

# Simulate the continuous exponential signal
continuous_exponential = simulate_continuous_exponential(time, amplitude,
coefficient)

# Simulate the discrete exponential signal
discrete_exponential = simulate_discrete_exponential(num_samples, amplitude,
coefficient)

# Plot and display the continuous and discrete exponential signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_exponential)
plt.title('Continuous Exponential Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')

plt.subplot(2, 1, 2)
plt.stem(discrete_exponential)
plt.title('Discrete Exponential Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

```

**Subject: Digital Signal and Image Processing (01CT1513)**

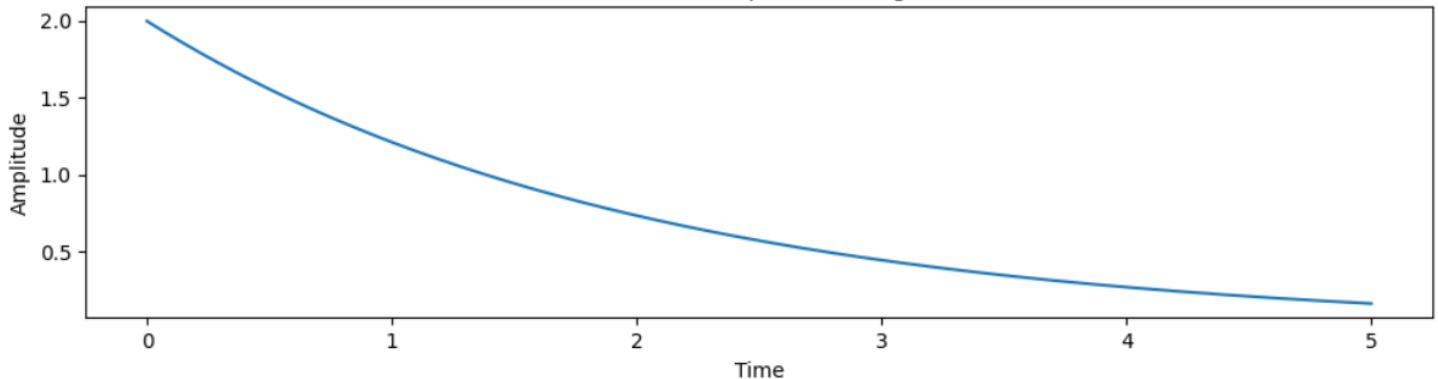
**Aim:** Simulate discrete time sequences.

**Experiment No: 1**

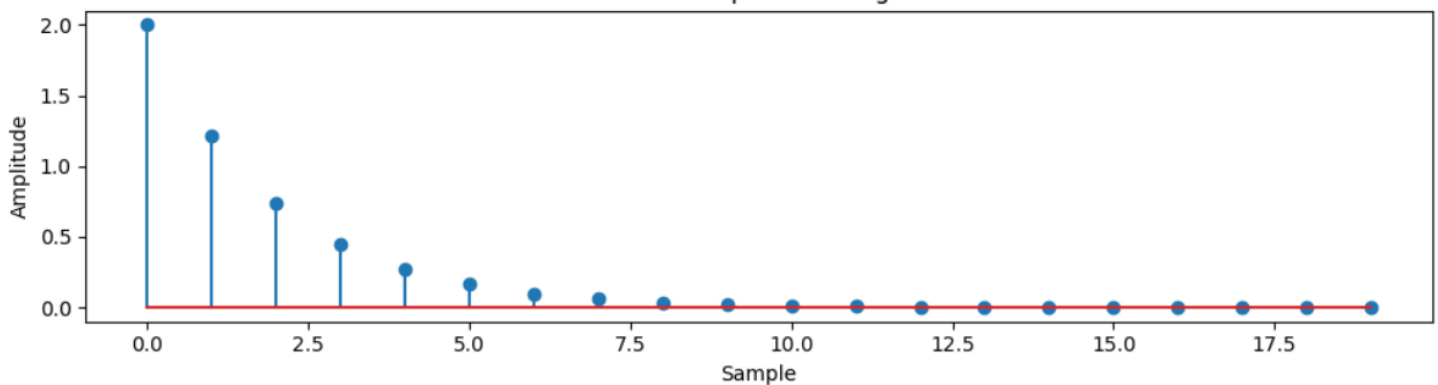
**Date:**

**Enrollment No: 92510133028**

Continuous Exponential Signal



Discrete Exponential Signal



```
import numpy as np
import matplotlib.pyplot as plt

def simulate_continuous_parabolic(time, coefficients):
    parabolic_signal = np.polyval(coefficients, time)
    return parabolic_signal

def simulate_discrete_parabolic(num_samples, coefficients):
    parabolic_signal = np.polyval(coefficients, np.arange(num_samples))
    return parabolic_signal

# Define the time range for the continuous parabolic signal
time = np.linspace(-5, 5, 1000) # Time range from -5 to 5

# Define the number of samples and coefficients for the discrete parabolic signal
num_samples = 20 # Number of samples
```



|                                                                                                                                                                                                                          |                                                                                                                                              |                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b><br>Marwadi Chandarana Group  | <b>Marwadi University</b><br><b>Faculty of Engineering &amp; Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Digital Signal and Image Processing (01CT1513)</b>                                                                                                                                                           | <b>Aim:</b> Simulate discrete time sequences.                                                                                                |                                   |
| <b>Experiment No: 1</b>                                                                                                                                                                                                  | <b>Date:</b>                                                                                                                                 | <b>Enrollment No: 92510133028</b> |

```

coefficients = [1, 2, 1] # Coefficients of the parabolic signal

# Simulate the continuous parabolic signal
continuous_parabolic = simulate_continuous_parabolic(time, coefficients)

# Simulate the discrete parabolic signal
discrete_parabolic = simulate_discrete_parabolic(num_samples, coefficients)

# Plot and display the continuous and discrete parabolic signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_parabolic)
plt.title('Continuous Parabolic Signal')
plt.xlabel('Time')
plt.ylabel('Amplitude')

plt.subplot(2, 1, 2)
plt.stem(discrete_parabolic)
plt.title('Discrete Parabolic Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()

# Save the parabolic signal arrays (optional)
# np.savetxt('continuous_parabolic.txt', continuous_parabolic, delimiter=',') #
np.savetxt('discrete_parabolic.txt', discrete_parabolic, delimiter=',')

```

**Subject: Digital Signal and Image Processing (01CT1513)**

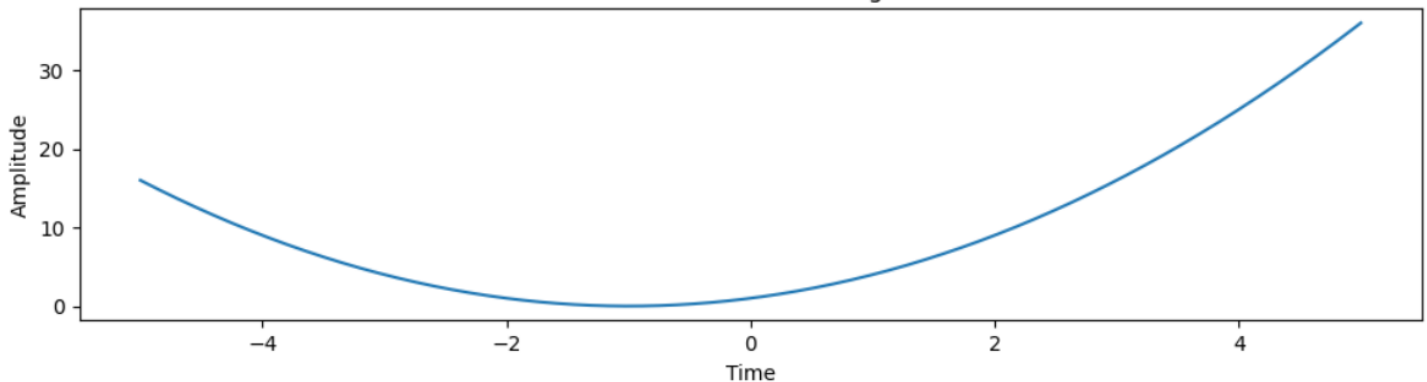
**Aim:** Simulate discrete time sequences.

**Experiment No: 1**

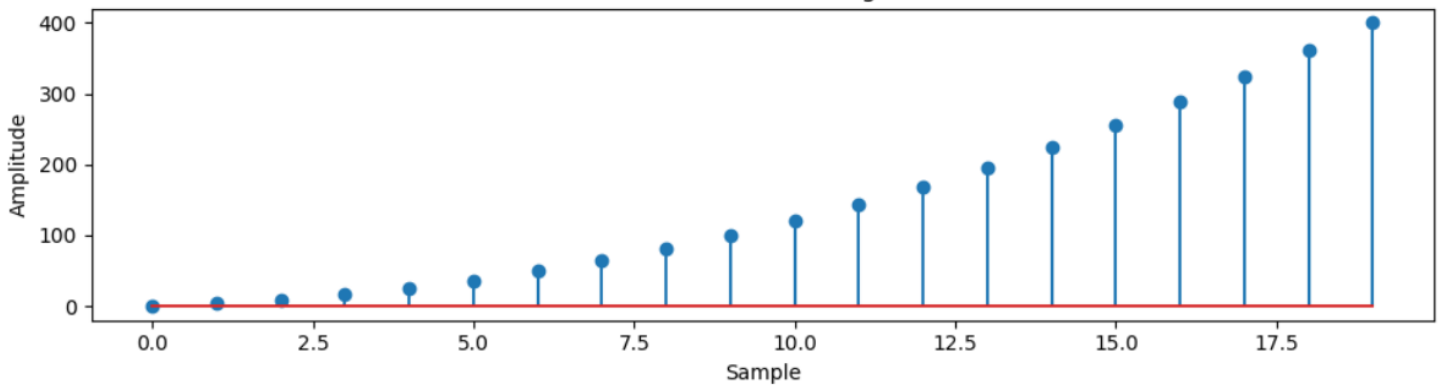
**Date:**

**Enrollment No: 92510133028**

Continuous Parabolic Signal



Discrete Parabolic Signal



```
frequency = 2 # Frequency of the sine wave in Hz
phase = 0 # Phase angle of the sine wave in radians
# Simulate the continuous sine wave signal
continuous_sine_wave = simulate_continuous_sine_wave(time, amplitude, frequency,
phase)
# Simulate the discrete sine wave signal
discrete_sine_wave = simulate_discrete_sine_wave(num_samples,
sampling_frequency, amplitude, frequency, phase)
# Plot and display the continuous and discrete sine wave signals
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.plot(time, continuous_sine_wave)
plt.title('Continuous Sine Wave Signal')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.subplot(2, 1, 2)
```

**Subject: Digital Signal and Image Processing (01CT1513)**

**Aim:** Simulate discrete time sequences.

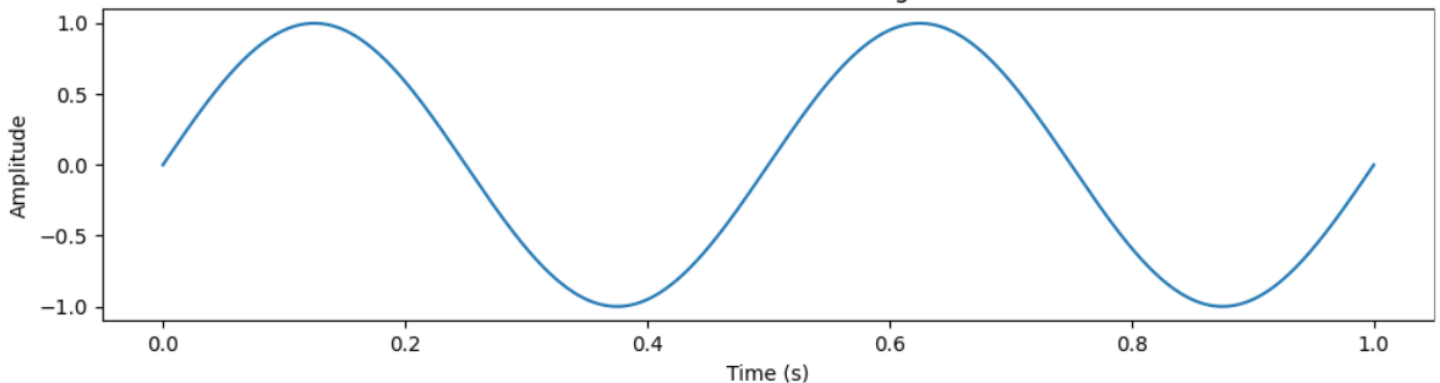
**Experiment No: 1**

**Date:**

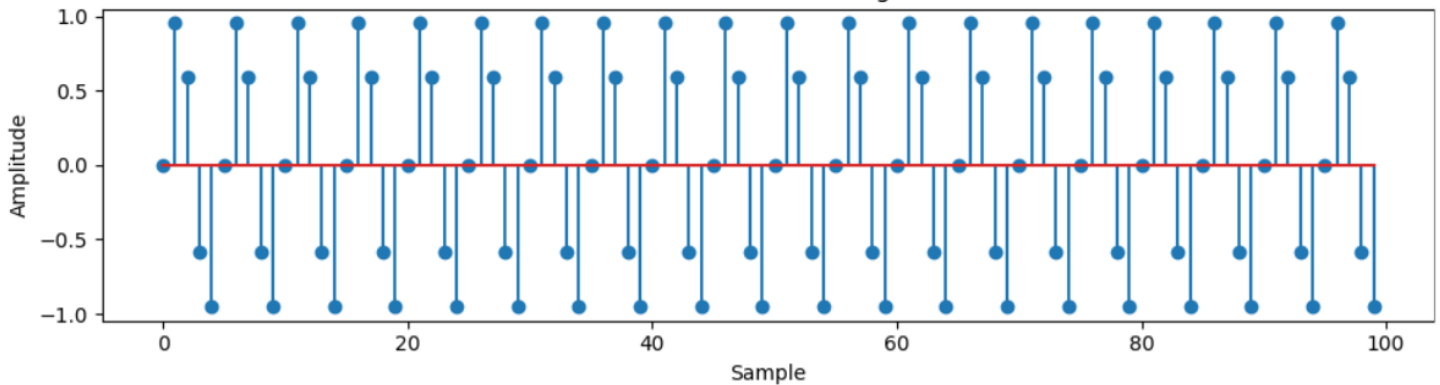
**Enrollment No: 92510133028**

```
plt.stem(discrete_sine_wave)
plt.title('Discrete Sine Wave Signal')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.tight_layout()
plt.show()
# Save the sine wave signal arrays (optional)
# np.savetxt('continuous_sine_wave.txt', continuous_sine_wave, delimiter=',') #
np.savetxt('discrete_sine_wave.txt', discrete_sine_wave, delimiter=',')
```

Continuous Sine Wave Signal



Discrete Sine Wave Signal



|                                                                                                                                                                                                                          |                                                                                                                                              |                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b><br>Marwadi Chandarana Group  | <b>Marwadi University</b><br><b>Faculty of Engineering &amp; Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Digital Signal and Image Processing (01CT1513)</b>                                                                                                                                                           | <b>Aim:</b> Simulate discrete time sequences.                                                                                                |                                   |
| <b>Experiment No: 1</b>                                                                                                                                                                                                  | <b>Date:</b>                                                                                                                                 | <b>Enrollment No: 92510133028</b> |

```
import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
    y = np.zeros_like(time)
    y[time >= 0] = 1
    y[time >= 1] += 1
    y[time >= -5] += 3
    return y
# Define the time range
time = np.linspace(-10, 10, 1000)
# Simulate the function
function_values = simulate_function(time)
# Plot and display the function
plt.plot(time, function_values)
plt.title('Function  $y(t) = u(t) + u(t-1) + 3*u(t+5)$ ')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 5.5])
plt.grid(True)
plt.show()
```

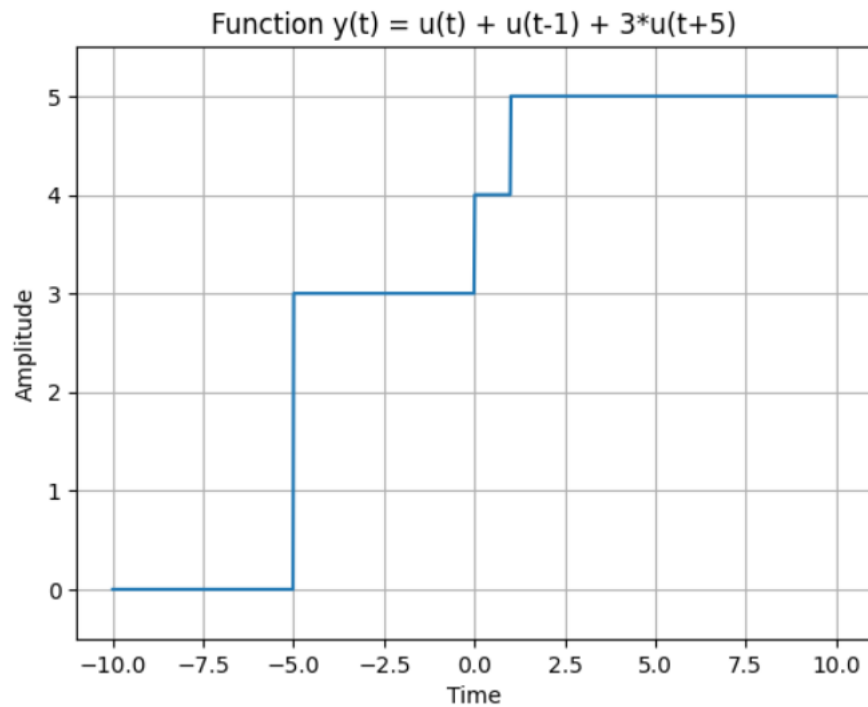
**Subject: Digital Signal and Image Processing (01CT1513)**

**Aim:** Simulate discrete time sequences.

**Experiment No: 1**

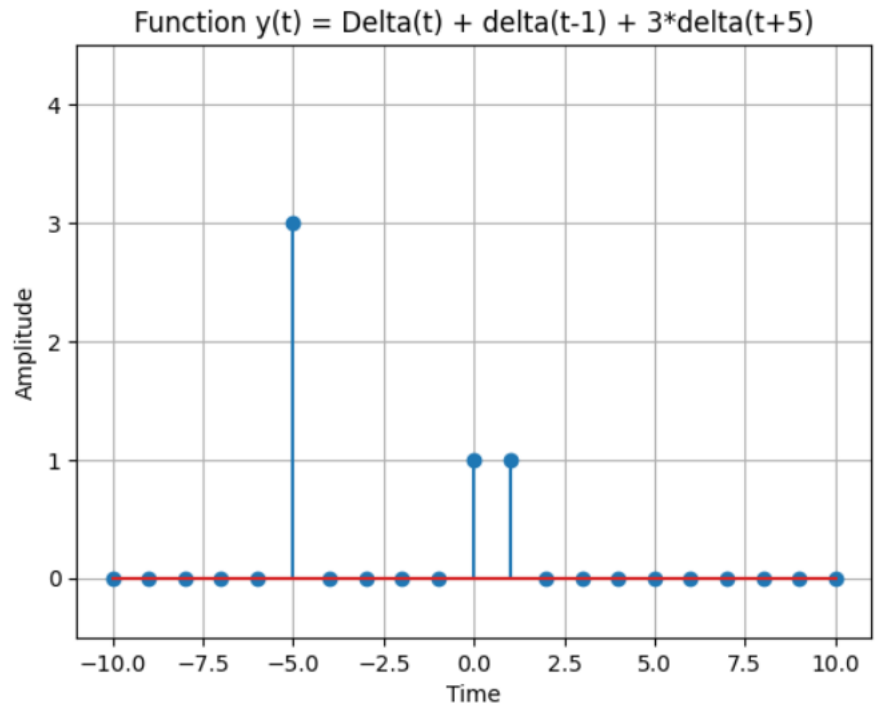
**Date:**

**Enrollment No: 92510133028**



```
import numpy as np
import matplotlib.pyplot as plt
def simulate_function(time):
    y = np.zeros_like(time)
    y[time == 0] = 1
    y[time == 1] += 1
    y[time == -5] += 3
    return y
# Define the time range
time = np.arange(-10, 11)
# Simulate the function
function_values = simulate_function(time)
# Plot and display the function
plt.stem(time, function_values)
plt.title('Function y(t) = Delta(t) + delta(t-1) + 3*delta(t+5)')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.ylim([-0.5, 4.5])
plt.grid(True)
plt.show()
```

|                                                                                                                                        |                                                                                                                                              |                                   |
|----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
|  <b>Marwadi University</b><br>Marwadi Chandarana Group | <b>Marwadi University</b><br><b>Faculty of Engineering &amp; Technology</b><br><b>Department of Information and Communication Technology</b> |                                   |
| <b>Subject: Digital Signal and Image Processing (01CT1513)</b>                                                                         | <b>Aim:</b> Simulate discrete time sequences.                                                                                                |                                   |
| <b>Experiment No: 1</b>                                                                                                                | <b>Date:</b>                                                                                                                                 | <b>Enrollment No: 92510133028</b> |



**Conclusion:**

This experiment enhanced my knowledge of discrete-time sequences and their practical implementation through simulation. It provided valuable insight into the characteristics and behavior of various discrete-time signals, improving my understanding of signal processing fundamentals.

**Github:**

[https://github.com/keshvi1234/DSIP\\_Exp](https://github.com/keshvi1234/DSIP_Exp)